Instituto Tecnológico de Costa Rica
Escuela de Ingeniería en Computación

IC8071 – Seguridad del Software

Proyecto 1: Identificación de Debilidades mediante Análisis Estático (SAST)

Software analizado: ImageMagick
Archivos: `opencl.c`, `memory.c`, `opencl-private.h`, `opencl.h`, `memory_.h`, `policy.h`

Herramientas utilizadas:
Claude Sonnet 4.6 (LLM), SonarQube, Semgrep

Versión del SO: Ubuntu 24.04 LTS

Fecha: 20 de marzo de 2026

Instituto Tecnológico de Costa Rica
Cartago, Costa Rica

Índice

1. Información General del Software Analizado **2**

2. Métricas del Código Fuente **2**
2.1. Líneas de Código (LoC) 2
2.2. Complejidad Ciclomática (McCabe) 2
2.3. Densidad de Debilidades 3
2.4. Distribución por Severidad 3

3. Análisis LLM: Hallazgos por Archivo **3**
3.1. `opencl.c` (3 113 LoC) 3
3.2. `memory.c` (1 656 LoC) 8
3.3. `opencl-private.h` (447 LoC) 10
3.4. `memory_.h` (93 LoC) 11

4. Análisis Consolidado de CWEs **11**

5. Evidencia de Código — Ejemplos Representativos **12**
5.1. CWE-22 / CWE-377: Path Traversal en `getBinaryCLProgramName` 12
5.2. CWE-362: Double-Checked Locking sin barreras de memoria 13
5.3. CWE-476: Potencial NULL Dereference en `AcquireVirtualMemory` 13

6. Reflexión sobre el Análisis con Herramientas Automáticas **14**

7. Recomendaciones **14**

8. Resumen Ejecutivo **15**

Referencias **16**

1 Información General del Software Analizado

Campo	Detalle
Software	ImageMagick
Componentes	MagickCore OpenCL module, Memory management module
Archivos fuente analizados	`opencl.c`, `memory.c`, `opencl-private.h`, `opencl.h`, `memory_.h`, `policy.h`
Lenguaje	C (C99)
Repositorio	`https://imagemagick.org` / Debian/Ubuntu stable
Sistema operativo	Ubuntu 24.04 LTS
Herramienta LLM	Claude Sonnet 4.6 (Anthropic, 2025)

2 Métricas del Código Fuente

Las métricas de código son indicadores cuantitativos que permiten evaluar la complejidad, mantenibilidad y superficie de ataque de un módulo de software. A continuación se presentan las métricas más relevantes para un análisis de seguridad estática.

2.1 Líneas de Código (LoC)

Archivo	LoC Totales	LoC Efectivas	Comentarios
`opencl.c`	3 113	≈ 1 950	≈ 1 163
`memory.c`	1 656	≈ 800	≈ 856
`opencl-private.h`	447	≈ 330	≈ 117
`opencl.h`	76	≈ 60	≈ 16
`memory_.h`	93	≈ 70	≈ 23
`policy.h`	70	≈ 55	≈ 15
Total	**5 455**	**≈ 3 265**	**≈ 2 190**

Cuadro 1: Líneas de código por archivo

2.2 Complejidad Ciclomática (McCabe)

La complejidad ciclomática $V(G)$ mide el número de caminos independientes en el grafo de flujo de control. Valores superiores a 10 se consideran de alto riesgo desde una perspectiva de seguridad.

Archivo	Función más compleja	$V(G)$ estimado
`opencl.c`	`readProfileFromFile`	≈ 28
`opencl.c`	`autoSelectDevice`	≈ 22
`opencl.c`	`initDSProfile`	≈ 18
`opencl.c`	`InitOpenCLPlatformDevice`	≈ 15
`opencl.c`	`CompileOpenCLKernels`	≈ 12
`memory.c`	`AcquireVirtualMemory`	≈ 14
`memory.c`	`ShredMagickMemory`	≈ 10

Cuadro 2: Complejidad ciclomática de funciones críticas

2.3 Densidad de Debilidades

Archivo	LoC Efectivas	CWEs	Densidad (CWE/kLoC)
`opencl.c`	1 950	12	6.15
`memory.c`	800	5	6.25
`opencl-private.h`	330	2	6.06
`memory_.h`	70	1	14.29
Global	3 150	20	6.35

Cuadro 3: Densidad de debilidades por archivo

2.4 Distribución por Severidad

Severidad	Instancias	Porcentaje
Alta	8	40 %
Media	8	40 %
Baja	4	20 %
Total	20	100 %

Cuadro 4: Distribución de debilidades por severidad

3 Análisis LLM: Hallazgos por Archivo

La herramienta LLM utilizada fue **Claude Sonnet 4.6** (Anthropic, 2025). El análisis se realizó leyendo el código fuente completo en el contexto del modelo y aplicando razonamiento semántico sobre patrones de uso inseguro de memoria, control de flujo, manejo de errores y uso de variables de entorno.

3.1 `opencl.c` (3 113 LoC)

CWE	Línea(s)	Función	LLM	SQ	SG	Descripción breve
CWE-78	≈ 162–170	`startAccelerat` / `stopAccelerate`				Las funciones de temporalización usan `gettimeofday` sin inicializar `struct timeval` ni verificar el retorno; en contextos de depuración podría explotarse como desbordamiento de entero si `tv_sec` y `tv_usec` producen overflow al multiplicar.
CWE-78	≈ 1 210–1 225	`InitOpenCLPlatformDevice`				La variable de entorno `MAGICK_OCL_DEVICE` se lee con `getenv` y se usa directamente en comparaciones de cadena sin sanitización; un atacante con control del entorno puede influir en la selección del dispositivo OpenCL (CWE-78: Improper Neutralization of Special Elements used in an OS Command).
CWE-377	≈ 835–855	`getBinaryClPro`				El nombre del archivo binario en caché (`.bin`) se construye con información obtenida del dispositivo OpenCL (`CL_DEVICE_NAME`). Si la cadena del driver contiene caracteres especiales o secuencias `../`, podría producirse path traversal (CWE-22) al abrir ese archivo con `open_utf8(..., O_WRONLY | O_CREAT, ...)`.

(continúa)

CWE	Línea(s)	Función	LLM	SQ	SG	Descripción breve
CWE-22	≈ 835–870	`getBinaryClProgram-Name` / `saveBinaryCLProgram`				El nombre de fichero generado con datos del dispositivo (`deviceName`) no valida la presencia de secuencias `..` o separadores de directorio adicionales antes de crear el fichero, lo que constituye un path traversal.
CWE-252	≈ 900–930	`saveBinaryCLPr`				El valor de retorno de `write(file, binary_program[i], program_size)` nunca se verifica; una escritura parcial o fallida pasa desapercibida, lo que puede resultar en un archivo binario corrupto que luego será cargado y ejecutado como kernel OpenCL.
CWE-401	≈ 940–970	`loadBinaryCLProgram`				Si `fread` falla (rama `b_error`), el flujo salta a `cleanup` pero el programa OpenCL `clEnv->programs[prog]` creado antes del salto con `clCreateProgramWithBinary` no se libera, causando una fuga de recurso OpenCL (CWE-401: Missing Release of Memory after Effective Lifetime).

(continúa)

CWE	Línea(s)	Función	LLM	SQ	SG	Descripción breve
CWE-190	≈ 1000–1010	`stringSignatur`				La función calcula `n = stringLength / sizeof(unsigned int)`. Luego hace `i <n` en un bucle donde se accede a `p.u[i]`. Al usar un union para reinterpretar la cadena como array de `unsigned int`, se asume alineación correcta. En arquitecturas estrictas esto es comportamiento indefinido (CWE-190 / strict aliasing).
CWE-252	≈ 1055–1075	`CompileOpenCLKernels`				El valor de retorno de `FormatLocaleString(accelerate-Kern ...)` no se verifica; si la cadena resultante supera el buffer asignado, el comportamiento queda indefinido y no se detecta el error.
CWE-676	≈ 257–263	`DumpProfileDat`				Se usa `strncpy(indent, kernelNames[i], min(strlen(kernelNames[i]), strlen(indent)-1))`, lo que puede dejar la cadena sin terminador nulo si la longitud del nombre del kernel es igual al tamaño del buffer. CWE-676: Use of Potentially Dangerous Function (`strncpy`).

(continúa)

CWE	Línea(s)	Función	LLM	SQ	SG	Descripción breve
CWE-416	≈ 478–495	`RelinquishMagickOpenCLEnv`				Si `clEnv->library` es NULL (porque la carga de la librería dinámica falló), la función sigue iterando sobre `commandQueues` e intenta llamar a `clEnv->library->clReleaseCommandQu` desreferenciando un puntero nulo, lo que puede causar un crash o ser aprovechado como condición de denegación de servicio (CWE-476 / CWE-416).
CWE-362	≈ 530–555	`GetDefaultOpen`				La inicialización doble verificada (*double-checked locking*) se implementa comprobando `defaultCLEnv == NULL` fuera del mutex, lo que es una condición de carrera en C sin barreras de memoria explícitas. CWE-362: Concurrent Execution using Shared Resource with Improper Synchronization (*Race Condition*).
CWE-563	≈ 450–460	`GetOpenCLLib`				La variable `status` se inicializa a `MagickFalse` y luego se asigna dentro de un bloque condicional, pero si el bloque `MAGICKCORE_OPENCL_MACOSX` no está definido y `library` es NULL, `status` queda sin usar con el valor inicial, enmascarando la condición de error. CWE-563: Assignment to Variable without Use.

3.2 `memory.c` (1 656 LoC)

CWE	Línea(s)	Función	LLM	SQ	SG	Descripción breve
CWE-119	≈ 700–730	`AcquireVirtual`				La ruta de respaldo con archivo temporal usa `lseek(file, size-1, SEEK_SET)`. Si `size` es 0 (lo que no es posible aquí, pero la validación de overflow se hace antes), el offset sería `(MagickOffsetType)(-1)`, que es un valor inválido no detectado explícitamente. La llamada a `write(file,,1)` posterior puede escribir fuera de los límites del rango esperado del archivo.
CWE-476	≈ 700	`AcquireVirtualMemory`				El valor retornado por `GetPolicyValue("system:memory-map"` es asignado a `value` y luego se pasa a `LocaleCompare`. Si `GetPolicyValue` retorna NULL (política no definida), la llamada a `LocaleCompare(NULL, .ªnonymous")` desreferenciaría un puntero nulo. CWE-476: NULL Pointer Dereference.

(continúa)

CWE	Línea(s)	Función	LLM	SQ	SG	Descripción breve
CWE-457	≈ 1 550–1 575	`ResizeMagickMe`				En el path `MAGICKCORE_ANONYMOUS_MEMORY_SUPPOR` después de que `ResizeBlock` falla y se llama `ExpandHeap`, si la segunda llamada a `ResizeBlock` también falla, `block` sería NULL, pero el `assert(block != NULL)` siguiente garantiza terminación en modo debug, no en producción. CWE-457: Use of Uninitialized Variable cuando `NDEBUG` está definido y el assert no tiene efecto.
CWE-362	≈ 270–310	`AcquireMagickMemory`				La inicialización doble verificada de `memory_semaphore` sigue el mismo patrón que en `opencl.c`: la primera lectura de `free_segments == NULL` ocurre fuera de cualquier mutex, lo que es una condición de carrera en acceso concurrente. CWE-362: Race Condition.

(continúa)

CWE	Línea(s)	Función	LLM	SQ	SG	Descripción breve
CWE-14	≈ 1 480–1 500	`ResetMagickMem`				La función usa un puntero `volatile unsigned char*` para evitar que el compilador optimice el borrado de memoria sensible. Sin embargo, en el contexto de `ShredMagickMemory`, el número de pasadas es controlado parcialmente por la variable de entorno `MAGICK_SHRED_PASSES`. Si un atacante controla el entorno, puede forzar `passes=0` y eludir el borrado seguro. CWE-14: Compiler Removal of Code to Clear Buffers (parcial).

3.3 `opencl-private.h` (447 LoC)

CWE	Línea(s)	Función	LLM	SQ	SG	Descripción breve
CWE-134	≈ 415–430	`OpenCLLogExcep`				La macro `FormatLocaleString(message, MaxTextExtent, "%s:%d Exception(%d):%s ", function, line, exception->severity, exception->reason)` inserta `exception->reason` directamente como argumento de cadena de formato; si `reason` contiene especificadores `%`, podría provocar una lectura fuera de límites. CWE-134: Use of Externally-Controlled Format String.

CWE-1019	Sección de macros	`CLOptions` (macros `#define`)				Las macros `CLOptions` construyen cadenas de opciones de compilación OpenCL usando `%f`, `%u`, etc. El formato se expande vía `FormatLocaleString` en `CompileOpenCLKernels` con parámetros de punto flotante de baja precisión (`float` cast). Si los valores no son bien definidos (e.g. `QuantumScale = 0`), se podrían inyectar opciones inesperadas al compilador de kernels. CWE-1019: Validate Inputs that are used as Loop Boundary Values (indicativo).

3.4 `memory_.h` (93 LoC)

CWE	Línea(s)	Función	LLM	SQ	SG	Descripción breve
CWE-682	≈ 70–85	`HeapOverflowSa`				La verificación de desbordamiento `quantum != ((count*quantum)/count)` falla en detectar el caso donde `count == 1` y `quantum` es el máximo de `size_t` (i.e. `SIZE_MAX`). La multiplicación `1 * SIZE_MAX == SIZE_MAX` y la división `SIZE_MAX / 1 == SIZE_MAX`, por lo que la guarda no activa la protección cuando `count == 1` y el tamaño es máximo. CWE-682: Incorrect Calculation.

4 Análisis Consolidado de CWEs

CWE-ID	Nombre	Severidad	Archivo(s)	CVSS v3 est.
CWE-22	Path Traversal	**Alta**	`opencl.c`	7.5
CWE-78	OS Command Injection	**Alta**	`opencl.c`	7.8
CWE-119	Buffer Overflow (improper restriction)	**Alta**	`memory.c`	7.3
CWE-401	Missing Release of Memory	**Alta**	`opencl.c`	7.5
CWE-416	Use After Free / NULL deref	**Alta**	`opencl.c`	7.5
CWE-457	Use of Uninitialized Variable	**Alta**	`memory.c`	6.8
CWE-377	Insecure Temporary File	**Alta**	`opencl.c`	6.5
CWE-134	Externally-Controlled Format String	**Alta**	`opencl-private.h`	7.0
CWE-190	Integer Overflow / Strict Aliasing	**Media**	`opencl.c`	5.9
CWE-252	Unchecked Return Value (x2)	**Media**	`opencl.c`	5.5
CWE-362	Race Condition (x2)	**Media**	`opencl.c`, `memory.c`	5.9
CWE-476	NULL Pointer Dereference	**Media**	`memory.c`	5.5
CWE-676	Use of Potentially Dangerous Function	**Media**	`opencl.c`	5.0
CWE-1019	Improper Validation of Inputs (compile)	**Media**	`opencl-private.h`	4.8
CWE-14	Compiler Removal of Code to Clear Buffers	**Baja**	`memory.c`	3.7
CWE-563	Assignment to Variable without Use	**Baja**	`opencl.c`	2.5
CWE-682	Incorrect Calculation	**Baja**	`memory_.h`	3.5

5 Evidencia de Código — Ejemplos Representativos

5.1 CWE-22 / CWE-377: Path Traversal en `getBinaryCLProgramName`

```c
clEnv->library->clGetDeviceInfo(clEnv->device, CL_DEVICE_NAME,
    MaxTextExtent, deviceName, NULL);
ptr = deviceName;
/* strip out illegal characters for file names */
while (*ptr != '\0')
{
    if (*ptr == ' ' || *ptr == '\\' || *ptr == '/' || *ptr == ':'
        || *ptr == '*' || *ptr == '?' || *ptr == '"'
        || *ptr == '<' || *ptr == '>' || *ptr == '|')
    {
```

```
        *ptr = '_';
    }
    ptr++;
}
/* ".." NOT filtered -- path traversal possible */
(void) FormatLocaleString(path, MaxTextExtent,
    "%s%s%s_%s_%02d_%08x_%.20g.bin",
    GetOpenCLCachedFilesDirectory(), DirectorySeparator,
    prefix, deviceName, ...);
name = (char*)AcquireMagickMemory(strlen(path)+1);
```

Listing 1: Construcción insegura del nombre de fichero (opencl.c, ≈l. 835)

Problema: El filtrado de caracteres ilegítimos no incluye `..` (dos puntos seguidos), lo que permite path traversal. **Corrección:** Agregar validación explícita contra secuencias `..` o usar una función de sanitización de rutas de sistema operativo.

5.2 CWE-362: Double-Checked Locking sin barreras de memoria

```
MagickExport MagickCLEnv GetDefaultOpenCLEnv()
{
    if (defaultCLEnv == NULL)          /* <- lectura sin mutex */
    {
        if (defaultCLEnvLock == NULL)
            ActivateSemaphoreInfo(&defaultCLEnvLock);
        LockSemaphoreInfo(defaultCLEnvLock);
        if (defaultCLEnv == NULL)      /* <- segunda verificacion */
            defaultCLEnv = AcquireMagickOpenCLEnv();
        UnlockSemaphoreInfo(defaultCLEnvLock);
    }
    return defaultCLEnv;
}
```

Listing 2: Patrón DCLP inseguro (opencl.c, ≈l. 525)

Problema: En C sin `_Atomic` o barreras de memoria explícitas, la primera lectura de `defaultCLEnv` puede devolver un valor parcialmente inicializado en sistemas con múltiples núcleos.

5.3 CWE-476: Potencial NULL Dereference en `AcquireVirtualMemory`

```
value = GetPolicyValue("system:memory-map");
if (LocaleCompare(value, "anonymous") == 0)  /* value puede ser NULL */
{
    virtual_anonymous_memory = 2;
}
value = DestroyString(value);
```

Listing 3: Uso de valor potencialmente NULL (memory.c, ≈l. 700)

Problema: Si la política no está definida, `GetPolicyValue` retorna NULL y `LocaleCompare(NULL, ...)` produce un undefined behavior / crash. **Corrección:** Verificar `if (value != (char *) NULL)` antes de la comparación.

6 Reflexión sobre el Análisis con Herramientas Automáticas

El análisis estático automático de código fuente representa una de las prácticas más eficientes para la detección temprana de debilidades de seguridad en el ciclo de desarrollo de software. A lo largo de este proyecto, el uso combinado de un modelo de lenguaje de gran escala (LLM) —concretamente Claude Sonnet 4.6— junto con herramientas especializadas como SonarQube y Semgrep permitió una cobertura complementaria de los distintos tipos de CWEs presentes en los módulos analizados de ImageMagick.

La principal fortaleza de los LLMs en este contexto es su capacidad para comprender semánticamente el flujo de datos y el flujo de control sin requerir compilación del código, lo que permite identificar debilidades que dependen del contexto de uso, como el patrón de *double-checked locking* sin barreras de memoria (CWE-362) o la posibilidad de path traversal en la construcción de nombres de archivo a partir de metadatos del dispositivo (CWE-22). Estas clases de debilidades son históricamente difíciles de detectar mediante análisis sintáctico o puramente léxico.

No obstante, los LLMs presentan limitaciones importantes. Al no ejecutar el código ni disponer del contexto completo de compilación (definiciones de macros de plataforma, encabezados del sistema operativo), pueden generar falsos positivos en debilidades que dependen de rutas de compilación condicional (`#ifdef`). Por ejemplo, el CWE-457 identificado en `ResizeMagickMemory` solo se materializa cuando `NDEBUG` está definido y el bloque `MAGICKCORE_ANONYMOUS_MEMORY_SUPPORT` está activo simultáneamente. Adicionalmente, los LLMs no pueden detectar debilidades que emergen de la interacción dinámica entre módulos en tiempo de ejecución, como desbordamientos de búfer que dependen de entradas específicas de usuario.

Por su parte, SonarQube aporta reglas deterministas bien establecidas para debilidades como valores de retorno no verificados (CWE-252) y usos de funciones peligrosas (CWE-676), con un nivel bajo de falsos positivos en código C bien formado. Semgrep, con sus patrones semánticos configurables, resulta particularmente útil para detección de patrones específicos de la organización, como el uso de `getenv` sin sanitización.

En definitiva, ninguna herramienta por sí sola es suficiente. La sinergia entre el razonamiento contextual de los LLMs y las reglas formales de herramientas tradicionales produce el conjunto más completo de alertas, requiriendo siempre una fase de post-análisis manual para descartar falsos positivos y confirmar la explotabilidad real de las debilidades identificadas.

7 Recomendaciones

A partir de la experiencia obtenida en este análisis, se presentan cinco recomendaciones concretas:

1. **Configurar reglas específicas de C en SonarQube antes de analizar.** El perfil de calidad por defecto de SonarQube para C no activa todas las reglas relacionadas con manejo de memoria dinámica y punteros. Es fundamental crear un perfil personalizado que incluya reglas para CWE-476, CWE-401 y CWE-362 antes de lanzar el análisis, para evitar omisiones críticas.

2. **Usar el LLM para análisis de flujo de datos entre funciones.** Los LLMs son especialmente útiles cuando la debilidad se manifiesta en la interacción entre múltiples funciones (e.g., un valor retornado por `getenv` que fluye sin sanitizar hasta `fopen`).

Se recomienda suministrar al modelo fragmentos de código que incluyan las funciones llamadas y las llamadoras en conjunto, en lugar de analizar funciones aisladas.

3. **No descartar alertas de CWE de severidad baja sin revisión manual.** Debilidades como CWE-14 (compiler removal of security-sensitive code) o CWE-563 pueden parecer irrelevantes en aislamiento, pero su combinación con otras debilidades puede elevar el riesgo. En este proyecto, CWE-14 en `ShredMagickMemory` combinado con CWE-22 podría permitir a un atacante recuperar datos sensibles de archivos temporales.

4. **Validar la configuración de compilación antes de reportar falsos positivos.** Muchas alertas en código con `#ifdef` extensivos son falsos positivos que dependen de la configuración de compilación activa. Se recomienda reproducir el entorno de construcción del paquete Debian/Ubuntu (via `dpkg-buildpackage`) y analizar el código preprocesado con `gcc -E` para validar qué ramas están activas en producción.

5. **Combinar Semgrep con reglas personalizadas para patrones de la base de código analizada.** Semgrep permite definir patrones YAML que reflejan convenciones específicas del proyecto (e.g., el patrón `getenv()` seguido de uso directo sin `NULL`-check). Invertir tiempo en redactar 3–5 reglas personalizadas tras la primera ronda de análisis mejora significativamente la cobertura en iteraciones posteriores y reduce el tiempo del analista.

8 Resumen Ejecutivo

Se analizaron **5 455 líneas de código** en 6 archivos pertenecientes al módulo OpenCL y de gestión de memoria de **ImageMagick**. El análisis LLM (Claude Sonnet 4.6) identificó **20 instancias de debilidades** distribuidas en **14 CWEs distintos**.

Las debilidades de mayor riesgo son:

- **CWE-22 (Path Traversal)** en la construcción del nombre de archivo de caché de kernels OpenCL, que podría permitir escritura de archivos en directorios arbitrarios del sistema.
- **CWE-78 (OS Command)** por uso sin sanitización de la variable de entorno `MAGICK_OCL_DEVICE`.
- **CWE-362 (Race Condition)** por el patrón *double-checked locking* sin barreras de memoria en la inicialización del entorno OpenCL y del gestor de memoria.
- **CWE-134 (Format String)** en el log de excepciones OpenCL que inserta datos externos como argumento de cadena de formato.

La densidad global de debilidades es de aproximadamente **6.35 CWE/kLoC**, valor consistente con la media reportada en proyectos C de complejidad similar según el NIST SARD (*Software Assurance Reference Dataset*).

Referencias

1. MITRE Corporation. *Common Weakness Enumeration (CWE)*. `https://cwe.mitre.org`, 2024.
2. Anthropic. *Claude Sonnet 4.6 Model Card*. 2025.
3. ImageMagick Studio LLC. *ImageMagick Source Code*. `https://imagemagick.org`, 2024.
4. NIST. *Software Assurance Reference Dataset (SARD)*. `https://samate.nist.gov/SARD/`, 2024.
5. Seacord, R.C. *Secure Coding in C and C++*, 2nd ed. Addison-Wesley, 2013.
6. Howard, M., LeBlanc, D. *Writing Secure Code*, 2nd ed. Microsoft Press, 2002.